

NumPy & Linear Algebra

The math foundation under everything else: vectors, matrices, broadcasting, dot products, and eigenvectors

Concept Guide 1 of 9 | Read before: *NumPy Zero-to-Master lab* | Companion to the *Zero-to-Hero* series

1. Vectors, Matrices, and Why Shape Matters

A vector is a list of numbers; a matrix is a grid of numbers. Everything in NumPy, and everything downstream (neural networks, embeddings, attention) is built from operations on these grids. The single most important habit: before doing any operation, know the shape of every array involved.

DIAGRAM — Ranks of arrays

scalar (0-D)	vector (1-D)	matrix (2-D)
5	[5, 2, 9]	[[1, 2, 3], [4, 5, 6]]
shape ()	shape (3,)	shape (2, 3)

MENTAL MODEL

A vector is one row of numbers. A matrix is a stack of vectors (rows), or equally, a stack of columns. Shape (2,3) always means '2 rows, 3 columns' — say it out loud every time you see a shape.

PRACTICE THIS → [NumPy_Zero_to_Master.ipynb, Chapters 1-3](#)

2. Matrix Multiplication

Matrix multiplication ($A @ B$) is NOT the same as multiplying elementwise. It combines rows of A with columns of B via dot products. This single operation underlies linear regression, every neural network layer, and the attention mechanism.

FORMULA — Matrix multiply

$$(A @ B)[i][j] = \text{sum over } k \text{ of } A[i][k] * B[k][j]$$

each output cell = dot product of a row of A with a column of B

WORKED EXAMPLE — A concrete 2x2 example

$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ $B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$

$(A @ B)[0][0] = 1*5 + 2*7 = 5 + 14 = 19$

$(A @ B)[0][1] = 1*6 + 2*8 = 6 + 16 = 22$

$(A @ B)[1][0] = 3*5 + 4*7 = 15 + 28 = 43$

$(A @ B)[1][1] = 3*6 + 4*8 = 18 + 32 = 50$

Result: $A @ B = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$

DIAGRAM — Shape rule

$(m \times n) @ (n \times p) = (m \times p)$
^inner dims must match^
the inner dimensions (n) 'cancel out'

COMMON MISUNDERSTANDINGS

- $A @ B$ is usually NOT the same as $B @ A$ (matrix multiplication is not commutative).
- The inner dimensions must match: $(2,3) @ (3,4)$ works and gives $(2,4)$; $(2,3) @ (2,4)$ does not work at all.
- Elementwise multiply ($A * B$, same shape required) is a completely different operation from matrix multiply ($A @ B$, inner shapes must match).

PRACTICE THIS → [NumPy_Zero_to_Master.ipynb, Chapter 11 \(Linear Algebra\)](#)

3. Broadcasting

Broadcasting lets NumPy combine arrays of different (but compatible) shapes without you manually copying data. It is the mechanism that makes 'subtract the mean of each column' a one-line operation instead of a loop.

FORMULA — The broadcasting rule

Compare shapes right-to-left.

Two dimensions are compatible if EQUAL, or if ONE of them is 1.

a missing dimension is treated as size 1

DIAGRAM — Broadcasting a bias vector across rows

```
data (4 rows x 3 cols)      +   bias (shape (3,))
[[10, 200, 1],              [1, 2, 3]
 [15, 220, 0],
 [ 9, 180, 1],
 [20, 260, 0]]
-> bias is 'stretched' virtually to match every row, no data is copied
```

COMMON MISUNDERSTANDINGS

- (4,3) combined with (4,) FAILS — the trailing dimensions (3 vs 4) don't match. You need (4,1) to broadcast a per-row value, not (4,).
- Always line shapes up right-to-left when reasoning about whether broadcasting will work.

PRACTICE THIS → [NumPy_Zero_to_Master.ipynb, Chapter 7](#)

4. Cosine Similarity, Norms, and the Dot Product

The dot product and vector norm (length) are the building blocks of similarity measures used throughout embeddings, search, and attention.

FORMULA — Dot product and norm

$$\begin{aligned} \mathbf{a} \cdot \mathbf{b} &= \sum(\mathbf{a}[\mathbf{i}] * \mathbf{b}[\mathbf{i}]) \\ ||\mathbf{a}|| &= \sqrt{\sum(\mathbf{a}[\mathbf{i}]^2)} \\ \text{cos_sim}(\mathbf{a}, \mathbf{b}) &= (\mathbf{a} \cdot \mathbf{b}) / (||\mathbf{a}|| * ||\mathbf{b}||) \end{aligned}$$

cosine similarity ranges from -1 (opposite) to 1 (identical direction)

WORKED EXAMPLE — Worked example

$\mathbf{a} = [1, 2, 3]$, $\mathbf{b} = [2, 4, 6]$ (b is exactly 2x a — same direction, different length)

$\mathbf{a} \cdot \mathbf{b} = 1*2 + 2*4 + 3*6 = 2 + 8 + 18 = 28$

$||\mathbf{a}|| = \sqrt{1+4+9} = \sqrt{14} = 3.742$

$||\mathbf{b}|| = \sqrt{4+16+36} = \sqrt{56} = 7.483$

$\text{cos_sim}(\mathbf{a}, \mathbf{b}) = 28 / (3.742 * 7.483) = 28 / 28.0 = 1.0$ (perfectly parallel)

MENTAL MODEL

Cosine similarity asks 'do these vectors point the same way?', completely ignoring length. This is why a short document and a long document about the same topic can still score as very similar — what matters is direction, not magnitude.

PRACTICE THIS → [NumPy_Zero_to_Master.ipynb Ch.11](#); [Embeddings_Search_Zero_to_Hero.ipynb Ch.3](#)

5. Eigenvectors and Eigenvalues (the PCA foundation)

An eigenvector of a matrix is a special direction that the matrix does not rotate — it only stretches (or shrinks) it, by a factor called the eigenvalue. This idea underlies PCA (dimensionality reduction) and the covariance structure used in statistics.

FORMULA — The eigenvector equation

$$A \mathbf{v} = \lambda \mathbf{v}$$

$\mathbf{v}$ is the eigenvector (a direction), λ is the eigenvalue (the stretch factor)

DIAGRAM — What it means geometrically

Most vectors get ROTATED and stretched by A .

Eigenvectors only get STRETCHED (same direction in, same direction out):

$$\mathbf{v} \xrightarrow{A} \lambda \mathbf{v} \quad (\text{same line, just longer/shorter})$$

MENTAL MODEL

In PCA, the eigenvectors of a dataset's covariance matrix are the 'directions of maximum spread' — the axes along which the data varies the most. The eigenvalues tell you HOW MUCH variance each direction captures, which is exactly what 'explained variance ratio' reports.

PRACTICE THIS → [Classical_ML_Zero_to_Hero.ipynb, Chapter 9 \(PCA\)](#)

6. Linear Systems and the Normal Equation Preview

Solving $Ax = b$ for x is one of the most common linear-algebra tasks in ML — it's literally how linear regression is solved in closed form (covered fully in the Classical ML guide).

FORMULA — Solving a linear system

$$\mathbf{A} \mathbf{x} = \mathbf{b} \Rightarrow \mathbf{x} = \mathbf{A}^{-1} \mathbf{b}$$

in practice, use a solver (like `linalg.solve`) rather than computing the inverse directly — it's more numerically stable

WORKED EXAMPLE — A tiny 2-equation system

$$2x + y = 5$$

$$x + 3y = 10$$

As a matrix: $\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix}$, $\mathbf{b} = [5, 10]$

Solving gives $x = 1$, $y = 3$ (check: $2 \cdot 1 + 3 = 5$ correct; $1 + 3 \cdot 3 = 10$ correct)

PRACTICE THIS → [NumPy_Zero_to_Master.ipynb Ch.11](#); [Classical_ML_Zero_to_Hero.ipynb Ch.2](#)